

AI trend-Generative AI and Prompts Engineering

Dr Hongqing (Harry) Yu

AI trend

- More XAI and more powerful AI chips supported Deep Learning Algorithms
- Generative AI (GenAI)
- Prompts Engineering

What is GenAI and Its Capabilities

- Generative AI refers to a subset of artificial intelligence technologies and models capable of generating new content that resembles human-created content. Unlike traditional AI, which is designed to recognize patterns, make predictions, or classify data, generative AI goes a step further by producing novel data instances, including text, images, videos, and music. The key feature of generative AI is its ability to understand and replicate the underlying patterns and structures of the input data, allowing it to create outputs that are often indistinguishable from those produced by humans.

- **Capabilities:**

Text Generation: Produces coherent and contextually relevant text passages, articles, stories, or even code.

Image Creation: Generates new images based on descriptions, combining elements in novel ways to produce artwork or photorealistic images.

Video Generation: Creates video clips from textual descriptions, simulating realistic scenarios or animating static images.

Music Composition: Composes music in various styles, generating melodies, harmonies, and rhythms that sound like they were created by human musicians.

Technologies Behind GenAI

- **Neural Networks**

A neural network is a series of algorithms modeled loosely after the human brain, designed to recognize patterns and interpret data. Neural networks consist of layers of nodes, or "neurons," which process input data and can learn to perform tasks without being explicitly programmed with rules.

- **Deep Learning**

Deep learning is a subset of machine learning involving neural networks with many layers. These deep neural networks can learn from a large amount of data, making them particularly effective for tasks that involve recognizing complex patterns, such as those found in images, sound, and language.

- **Transformer Architecture**

Transformer

- The transformer architecture, introduced in the paper "Attention is All You Need" by Vaswani et al, on 2020, has revolutionized the field of natural language processing (NLP) and generative AI. At its core, the transformer model eschews traditional sequential processing methods, such as recurrent neural networks (RNNs), in favor of a mechanism called **self-attention**. This allows the model to weigh the importance of different parts of the input data differently, making it highly efficient at capturing relationships and dependencies within the data, regardless of their distance from each other in the sequence.

Bank of the river.

v_1 v_2 v_3 v_4

REWEIGHT!

PROCESS

y_1 y_2 y_3 y_4

← better, more context

$$v_1 v_1 = w_{11}$$

$$v_1 v_2 = w_{12}$$

$$v_1 v_3 = w_{13}$$

$$v_1 v_4 = w_{14}$$

norm

$$w_{11}$$

$$w_{12}$$

$$w_{13}$$

$$w_{14}$$

sum to one

$$w_{11}v_1 + w_{12}v_2 + w_{13}v_3 + w_{14}v_4 = y_1$$

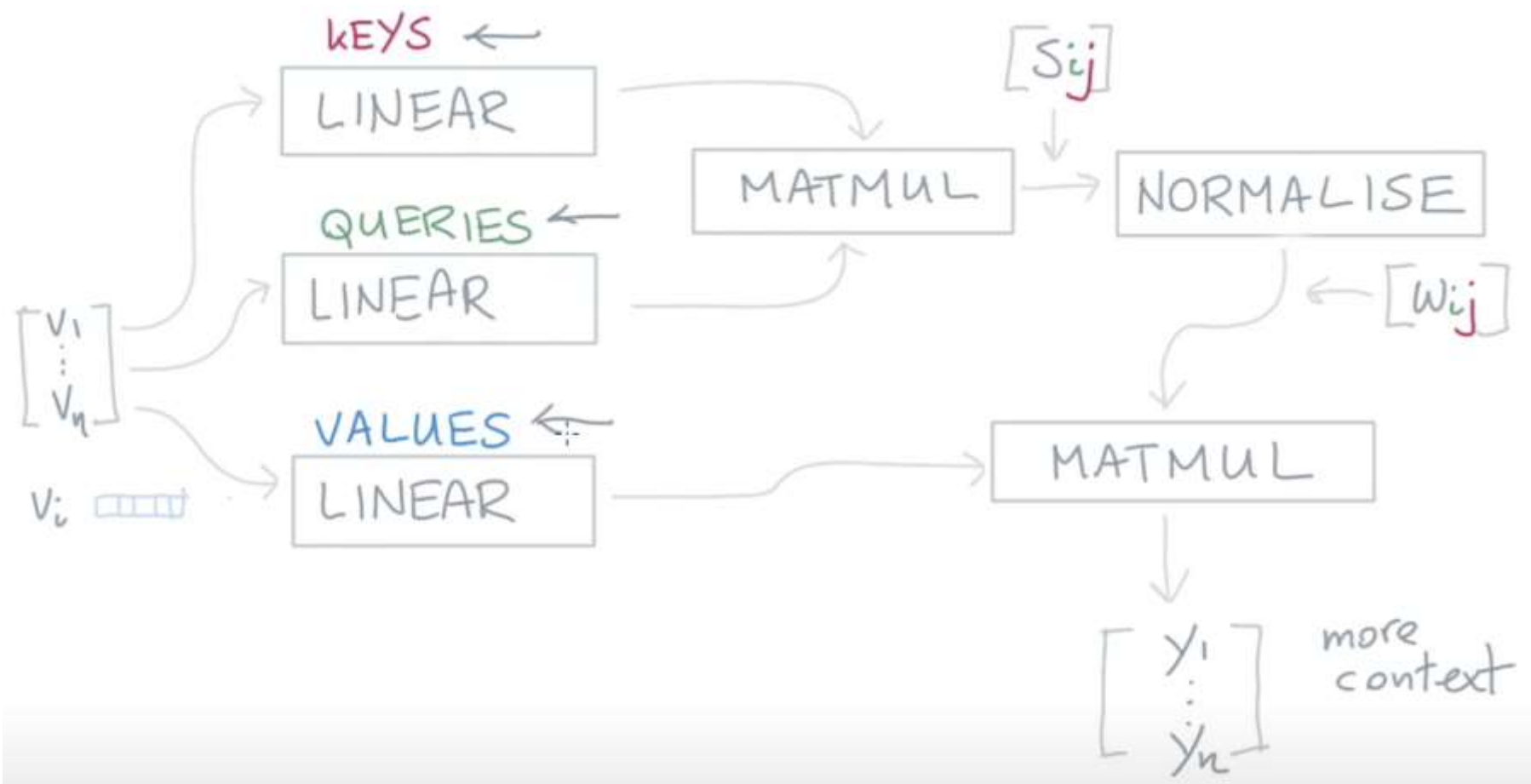
$$w_{21}v_1 + w_{22}v_2 + w_{23}v_3 + w_{24}v_4 = y_2$$

$$w_{31}v_1 + w_{32}v_2 + w_{33}v_3 + w_{34}v_4 = y_3$$

$$w_{41}v_1 + w_{42}v_2 + w_{43}v_3 + w_{44}v_4 = y_4$$

- I've not trained any weights
- Order has no influence
- Proximity has no influence
- Shape independant

SELF ATTENTION



https://colab.research.google.com/github/tensorflow/tensor2tensor/blob/master/tensor2tensor/notebooks/hello_t2t.ipynb#scrollTo=T7UJzFf6fmhp

Implementation Example

```
from numpy import array
from numpy import random
from numpy import dot
from scipy.special import softmax

# encoder representations of four different words
word_1 = array([1, 0, 0])
word_2 = array([0, 1, 0])
word_3 = array([1, 1, 0])
word_4 = array([0, 0, 1])

# stacking the word embeddings into a single array
words = array([word_1, word_2, word_3, word_4])

# generating the weight matrices
random.seed(42)
W_Q = random.randint(3, size=(3, 3))
W_K = random.randint(3, size=(3, 3))
W_V = random.randint(3, size=(3, 3))

# generating the queries, keys and values
Q = words @ W_Q
K = words @ W_K
V = words @ W_V
```

```
# scoring the query vectors against all key vectors
scores = Q @ K.transpose()

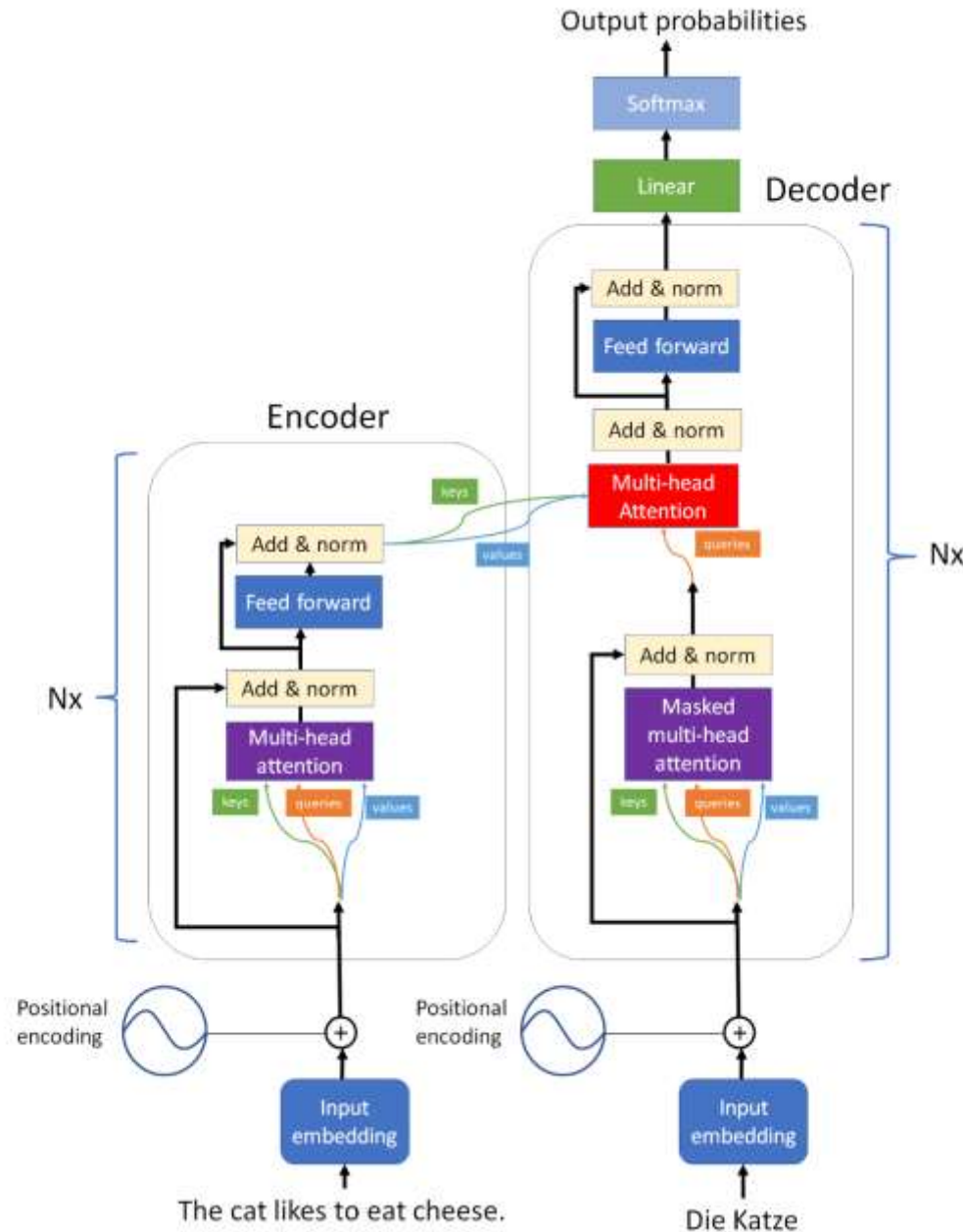
# computing the weights by a softmax operation
weights = softmax(scores / K.shape[1] ** 0.5, axis=1)

# computing the attention by a weighted sum of the value vectors
attention = weights @ V

print(attention)
```

```
[[0.98522025  1.74174051  0.75652026]
 [0.90965265  1.40965265  0.5       ]
 [0.99851226  1.75849334  0.75998108]
 [0.99560386  1.90407309  0.90846923]]
```


Transformer Attention



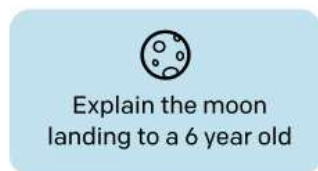
[Attention in Transformer | Towards Data Science](#)

Transformer architecture removes recurrence and replaces it with an attention mechanism, which uses queries to select the information(value) it needs, based on the label provided by the keys which learnt from previous training models. If keys, values and queries are generated from the same sequence, it is called self-attention. In cross attention, the queries are generated by a different sequence, than the key-value pairs. Multi-head attention helps the transformer to multiple things at the same time.

Step 1

Collect demonstration data, and train a supervised policy.

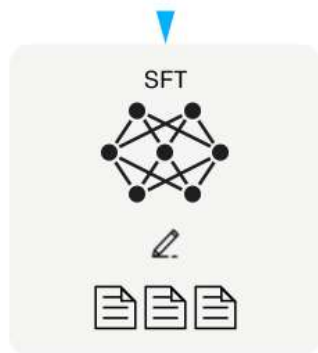
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



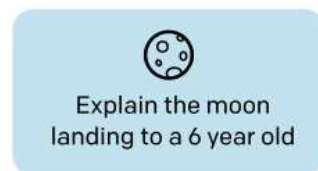
This data is used to fine-tune GPT-3 with supervised learning.



Step 2

Collect comparison data, and train a reward model.

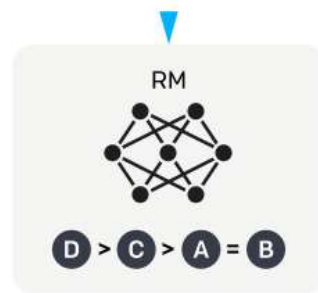
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



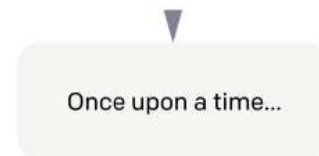
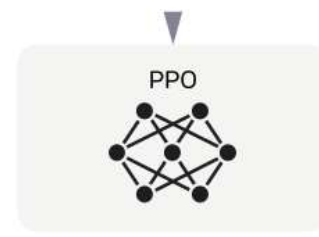
Step 3

Optimize a policy against the reward model using reinforcement learning.

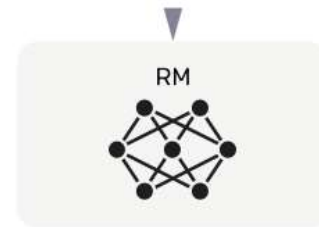
A new prompt is sampled from the dataset.



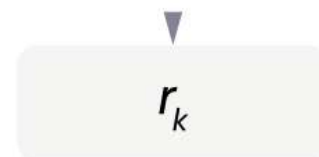
The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Prompts Engineering

- **What is Prompts Engineering?**

Prompts engineering is the process of designing and refining the input prompts given to AI models to elicit specific responses or outputs. This process is crucial for leveraging the full potential of generative AI, as the quality and specificity of a prompt can significantly influence the model's output. Effective prompts engineering can lead to more accurate, relevant, and creative results, whether the task involves generating text, images, code, or any other form of content.

Strategies and Techniques

1. Specificity and Clarity

Description: Being specific and clear in the prompt helps narrow down the AI's focus, leading to more accurate and relevant outputs.

Example Product Descriptions:

- **Prompt:** "Write a product description for a waterproof watch."
- **GPT Output:** "Introducing our latest innovation: a watch that keeps up with your adventurous lifestyle. This waterproof watch is designed to withstand depths of up to 50 meters, ensuring that whether you're swimming, diving, or caught in a downpour, your watch remains functional and stylish. With its sleek design and durable build, it's the perfect companion for all your escapades."
- **Refined Prompt:** "Write a short product description for a children's waterproof watch, highlighting its durability and fun design."
- **GPT Output:** "Meet the ultimate accessory for your little explorer: our Kid's Waterproof Watch! Designed with fun in mind, it features a vibrant, playful design that kids love, and parents appreciate its unmatched durability. Waterproof and tough, it can handle anything from puddle jumping to swimming lessons, making it the perfect timepiece for every adventure."

Strategies and Techniques

2. Role-Task-F

Context data, examples, to emotion and entropy

Table 4: Test accuracies on GSM8K. We show the instruction with the highest test accuracy for each scorer-optimizer pair.

Scorer	Optimizer / Source	Instruction position	Top instruction	Acc
<i>Baselines</i>				
PaLM 2-L	(Kojima et al., 2022)	A_begin	Let's think step by step.	71.8
PaLM 2-L	(Zhou et al., 2022b)	A_begin	Let's work this out in a step by step way to be sure we have the right answer.	58.8
PaLM 2-L		A_begin	Let's solve the problem.	60.8
PaLM 2-L		A_begin	(empty string)	34.0
text-bison	(Kojima et al., 2022)	Q_begin	Let's think step by step.	64.4
text-bison	(Zhou et al., 2022b)	Q_begin	Let's work this out in a step by step way to be sure we have the right answer.	65.6
text-bison		Q_begin	Let's solve the problem.	59.1
text-bison		Q_begin	(empty string)	56.8
<i>Ours</i>				
PaLM 2-L	PaLM 2-L-IT	A_begin	Take a deep breath and work on this problem step-by-step.	80.2
PaLM 2-L	PaLM 2-L	A_begin	Break this down.	79.9
PaLM 2-L	gpt-3.5-turbo	A_begin	A little bit of arithmetic and a logical approach will help us quickly arrive at the solution to this problem.	78.5
PaLM 2-L	gpt-4	A_begin	Let's combine our numerical command and clear thinking to quickly and accurately decipher the answer.	74.5
text-bison	PaLM 2-L-IT	Q_begin	Let's work together to solve math word problems! First, we will read and discuss the problem together to make sure we understand it. Then, we will work together to find the solution. I will give you hints and help you work through the problem if you get stuck.	64.4
text-bison	text-bison	Q_end	Let's work through this problem step-by-step:	68.5
text-bison	gpt-3.5-turbo	Q_end	Analyze the given information, break down the problem into manageable steps, apply suitable mathematical operations, and provide a clear, accurate, and concise solution, ensuring precise rounding if necessary. Consider all variables and carefully consider the problem's context for an efficient solution.	66.5
text-bison	gpt-4	Q_begin	Start by dissecting the problem to highlight important numbers and their relations. Decide on the necessary mathematical operations like addition, subtraction, multiplication, or division, required for resolution. Implement these operations, keeping in mind any units or conditions. Round off by ensuring your solution fits the context of the problem to ensure accuracy.	62.7

Notes

it GPT has trouble finding the context.

context is better now, much clear output

ing Chat GPT to act as a edu professional
ilted in better wording and more specific details

Strategies and Techniques

3. Incorporating Context

Description: Providing context within the prompt can guide the AI to understand the desired tone, style, or background information.

Example: "Draft an email to a client apologizing for a delayed shipment, ensuring to express empathy and offer a discount for future orders."

Strategies and Techniques

4. Few-shot, and Chain-of-thought Prompting

Description: These techniques involve crafting prompts that either provide no examples (zero-shot), a few examples (few-shot), or a logical reasoning path (chain-of-thought) to help the model understand and generate the desired output.

Example

- **Prompt:**

1. **Example 1:** "Translate 'Hello, how are you?' into French. -> 'Bonjour, comment ça va?'"
2. **Example 2:** "Translate 'I am learning to speak French.' into French. -> 'J'apprends à parler français.'"
3. **Task:** "Translate 'Can you help me?' into French."

- **AI Output:** "'Pouvez-vous m'aider?'"

Example

- **Prompt:** "You have 20 apples. You give 8 to your friend and eat 3 yourself. How many apples do you have left? Explain your reasoning step by step."
- **AI Output:** "You start with 20 apples. First, you give 8 apples to your friend, leaving you with $20 - 8 = 12$ apples. Then, you eat 3 of those apples, leaving you with $12 - 3 = 9$ apples. Therefore, you have 9 apples left."

This technique guides the model to process information in a sequential and logical manner, mimicking human-like reasoning to solve the problem.

Summarisation

- We will see and use lots of GenAI applications at very nearly future in our daily life
- Prompts engineering become crucial to enable us complete our job more efficiently, e.g. programming, write ppt and searching knowledge.
- GenAI will become the future system engine to drive the system's functionalities and GUIs to communicate with the users.